

Seals MES 系统软件保护方案

技术栈: Vue 3 + Ant Design Vue 4.x + Node.js/Express + SQL Server

编制日期: 2026-05-06

一、概述

Seals MES 系统作为企业核心制造执行系统，包含大量业务逻辑（MRP运算、MPS排产、生产调度等），部署到客户服务器后面临源码泄露、非法复制、未授权使用等风险。本文档从前端、后端、数据库、许可证四个维度，提供系统化的软件保护方案。

保护目标

风险类型	描述	危害等级
源码泄露	客户或第三方获取并阅读源代码	高
非法复制	将系统整体复制部署到其他服务器	高
未授权使用	超出授权范围使用（超额用户、超期使用）	中
核心算法窃取	提取MRP/MPS等核心计算逻辑	高
数据泄露	数据库被直接拷贝或导出	中

二、前端保护

2.1 代码混淆

使用 javascript-obfuscator 对打包后的 JavaScript 进行深度混淆：``javascript // vue.config.js 或 vite.config.ts 构建配置 const JavaScriptObfuscator = require('webpack-obfuscator'); module.exports = { configureWebpack: { plugins: [new JavaScriptObfuscator({ rotateStringArray: true, stringArray: true, stringArrayThreshold: 0.75, deadCodeInjection: true, deadCodeInjectionThreshold: 0.2, controlFlowFlattening: true, controlFlowFlatteningThreshold: 0.5, debugProtection: true, disableConsoleOutput: true }, [])] } }` `**效果**：变量名变为无意义字符、控制流被打乱、注入死代码、禁用浏览器调试。 **局限性**：前端代码本质上是公开的，混淆只能提高逆向门槛，无法完全阻止。 ### 2.2 关闭 Source Map `javascript // vite.config.ts export default defineConfig({ build: { sourcemap: false // 生产环境必须关闭 } })` `\*\*确保\*\*： - 生产构建不生成 .map 文件 - 构建产物中不包含 src/ 目录 - 环境变量通过 .env.production 注入，不硬编码密钥 ### 2.3 关键信息运行时注入 `javascript // 不要在前端代码中硬编码API地址和密钥 // 而是在构建时注入，运行时只读 // .env.production VITE_API_BASE_URL=/api VITE_APP_VERSION=__BUILD_VERSION__ // 运行时从

<meta> 标签或 window 全局变量读取后端下发的配置 `--- ## 三、后端保护（核心防线） ### 3.1 许可证验证机制（推荐首选） ##### 3.1.1 整体架构`

```

graph LR
    A[客户部署服务器] --> B[许可证验证中间件]
    A --> C[Express 应用]
    B --> D[授权服务器我方]
    C --> E[License Guard]
    E --> F[License Server]
    F --> G[机器指纹采集]
    G --> H[在线/离线双模式]
    H --> I[签发/续期/吊销]
  
```

3.1.2 机器指纹采集 绑定客户服务器硬件特征，确保 License 不可迁移：

```

`typescript // server/src/modules/license/machineFingerprint.ts import { execSync } from
'child_process'; import crypto from 'crypto'; export function getMachineFingerprint(): string { //
CPU序列号 const cpuId = execSync( 'wmic cpu get ProcessorId /value', { encoding: 'utf-8' }
).split('=')[1]?.trim() || ''; // 硬盘序列号（系统盘） const diskId = execSync( 'wmic diskdrive
get SerialNumber /value', { encoding: 'utf-8' } ).split('=')[1]?.trim() || ''; // MAC地址（第一个
物理网卡） const mac = execSync( 'wmic nic where "NetEnabled=true" get MACAddress /value', {
encoding: 'utf-8' } ).split('=')[1]?.trim() || ''; // 组合后SHA256 const raw =
`${cpuId}${diskId}${mac}`; return crypto.createHash('sha256').update(raw).digest('hex'); } ` #####

```

3.1.3 License 数据结构 `json { "licenseId": "LIC-20260506-001", "customer": "宁国睿信密封科技有限公司", "machineFingerprint": "a3f5b2c8...", "issuedAt": "2026-05-06", "expiresAt": "2027-05-06", "maxUsers": 50, "modules": ["MPS", "MRP", "生产执行", "采购", "仓储", "质量"], "features": { "mrpEnabled": true, "ganttEnabled": true, "appEnabled": false }, "signature": "RSA-SHA256签名值..." } ` #####

3.1.4 在线验证模式 `typescript // server/src/modules/license/licenseGuard.ts

```

import { Request, Response, NextFunction } from 'express'; let licenseCache: any = null; let
lastCheckTime = 0; const CHECK_INTERVAL = 4 * 60 * 60 * 1000; // 4小时 export async function
licenseGuard(req: Request, res: Response, next: NextFunction) { try { const now = Date.now(); //
缓存未过期，直接放行 if (licenseCache && now - lastCheckTime < CHECK_INTERVAL) { req.license =
licenseCache; return next(); } // 向授权服务器验证 const fingerprint = getMachineFingerprint();
const resp = await fetch('https://license.yourcompany.com/api/verify', { method: 'POST', headers:
{ 'Content-Type': 'application/json' }, body: JSON.stringify({ licenseId: config.licenseId,
fingerprint, timestamp: now }) }); const result = await resp.json(); if (result.valid) {
licenseCache = result.license; lastCheckTime = now; req.license = licenseCache; next(); } else {
res.status(403).json({ code: 403, message: result.reason || '许可证验证失败' }); } } catch (error)
{ // 授权服务器不可达时，走离线验证 const offlineResult = verifyOfflineLicense(); if
(offlineResult.valid) { req.license = offlineResult.license; next(); } else {
res.status(403).json({ code: 403, message: '无法连接授权服务器且离线许可证无效' }); } } } ` #####

```

3.1.5 离线验证模式 适用于客户内网隔离环境，使用 RSA 非对称签名： `typescript //

```

server/src/modules/license/offlineVerify.ts import crypto from 'crypto'; import fs from 'fs';
const PUBLIC_KEY = fs.readFileSync('license_public.pem', 'utf-8'); export function
verifyOfflineLicense() { try { const licPath = path.join(process.cwd(), 'license.lic'); const
licContent = fs.readFileSync(licPath, 'utf-8'); const license = JSON.parse(licContent); // 1. 验证
RSA签名 const { signature, ...payload } = license; const verify = crypto.createVerify('RSA-
SHA256'); verify.update(JSON.stringify(payload)); const sigValid = verify.verify(PUBLIC_KEY,
signature, 'base64'); if (!sigValid) { return { valid: false, reason: '许可证签名无效' }; } // 2.
验证机器指纹 const fingerprint = getMachineFingerprint(); if (license.machineFingerprint !==
fingerprint) { return { valid: false, reason: '许可证与当前服务器不匹配' }; } // 3. 验证到期时间
if (new Date(license.expiresAt) < new Date()) { return { valid: false, reason: '许可证已过期' }; }

```

```

return { valid: true, license }; } catch (e) { return { valid: false, reason: '许可证文件不存在或格式错误' }; } } `#### 3.1.6 License 签发工具（我方使用）`typescript // tools/license-issuer.ts
（仅在开发者本地运行，不部署）import crypto from 'crypto'; const PRIVATE_KEY =
fs.readFileSync('license_private.pem', 'utf-8'); function issueLicense(params: { customer: string;
fingerprint: string; expiresAt: string; maxUsers: number; modules: string[]; }) { const payload =
{ licenseId: LIC-${Date.now()}, customer: params.customer, machineFingerprint:
params.fingerprint, issuedAt: new Date().toISOString().slice(0, 10), expiresAt: params.expiresAt,
maxUsers: params.maxUsers, modules: params.modules }; const sign = crypto.createSign('RSA-
SHA256'); sign.update(JSON.stringify(payload)); const signature = sign.sign(PRIVATE_KEY,
'base64'); return { ...payload, signature }; } `#### 3.1.7 全局注册中间件`typescript //
server/src/app.ts import { licenseGuard } from './modules/license/licenseGuard'; // 在所有路由之前
注册 app.use('/api', licenseGuard); // 登录接口也需要验证（未通过License则无法登录） // 前端可增加
License到期提醒 `--- #### 3.2 核心模块字节码编译 使用 bytenode 将核心业务模块编译为 v8 字节
码，无法直接阅读源码：`bash npm install bytenode --save-dev `typescript // 编译脚本
tools/compile-core.ts import bytenode from 'bytenode'; import fs from 'fs'; // 编译核心算法模块
const coreFiles = [ 'server/src/modules/planning/mrp/mrp.controller.ts',
'server/src/modules/planning/mps/mps.controller.ts', 'server/src/modules/license/licenseGuard.ts'
]; for (const file of coreFiles) { bytenode.compileFile({ filename: file.replace('.ts', '.jsc'),
output: file.replace('.ts', '.jsc') }); } `typescript // 运行时加载字节码 const mrpController =
require('./mrp.controller.jsc');` **效果**：JSC 文件为 v8 字节码格式，无法用文本编辑器阅读，反编
译难度大。 --- #### 3.3 核心逻辑服务化（最强防护） 将 MRP 运算、MPS 计算等核心算法部署在我方服务
器，客户端只做 API 调用：
┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
│ 客户部署端 │ HTTPS API 调 │ 我方核心服务 │ │ ────────────> │ │ 基础CRUD操作 │ │ MRP运算引擎 │ │ 生产报工记录 │
└──────────────────┘ └──────────────────┘ └──────────────────┘ └──────────────────┘
←──────────────────┘ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
│ MPS排产算法 │ │ 库存出入库记录 │ 返回计算结果 │ 核心业务逻辑 │
└──────────────────┘ └──────────────────┘ └──────────────────┘ └──────────────────┘
- 客户部署端只包含 CRUD 和数据录入功能 - 核心算法永远
不离开我方服务器 - 客户完全无法接触核心代码 --- ## 四、数据库保护 #### 4.1 SQL Server TDE（透明数
据加密）`sql -- 创建数据库主密钥 CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword123!';
-- 创建证书 CREATE CERTIFICATE TDE_Cert WITH SUBJECT = 'TDE Certificate'; -- 创建数据库加密密钥
CREATE DATABASE ENCRYPTION KEY WITH ALGORITHM = AES_256 ENCRYPTION BY SERVER CERTIFICATE TDE_Cert;
-- 启用TDE ALTER DATABASE SEALSMES SET ENCRYPTION ON;` **效果**：数据库文件（.mdf/.ldf）和备份文
件均加密，直接拷贝无法附加。 #### 4.2 连接安全`typescript // 数据库连接配置 const config = { host:
'127.0.0.1', port: 1433, database: 'SEALSMES', username: process.env.DB_USER, // 不使用sa
password: process.env.DB_PASSWORD, options: { encrypt: true, // 加密连接 trustServerCertificate:
false, enableArithAbort: true } };` #### 4.3 账号权限最小化`sql -- 创建只读应用账号 CREATE LOGIN
mes_app WITH PASSWORD = 'AppPassword456!'; CREATE USER mes_app FOR LOGIN mes_app; -- 仅授予必要权
限（不使用sa） GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO mes_app; -- 禁止DDL操作 DENY
ALTER ON SCHEMA::dbo TO mes_app; DENY CREATE TABLE TO mes_app; DENY DROP TO mes_app;` --- ## 五、
部署加固 #### 5.1 Node.js 应用打包为可执行文件 使用 pkg 将应用打包为单一二进制文件：`bash npm
install pkg --save-dev npx pkg server/dist/app.js -t node18-win-x64 -o seals-mes-server.exe` **效
果**：源码嵌入可执行文件，不暴露 .js 文件。 #### 5.2 环境变量管理`bash # 不使用 .env 文件（容易被复
制），改用系统环境变量 # Windows 服务器设置： setx DB_PASSWORD "加密后的密码" /M setx LICENSE_ID
"LIC-20260506-001" /M setx JWT_SECRET "随机生成的密钥" /M` #### 5.3 进程守护与防调试`typescript
// server/src/app.ts - 防调试检测 if (process.env.NODE_ENV === 'production') { // 检测调试器附加

```

```
const { execSync } = require('child_process'); setInterval(() => { try { const result =
execSync('wmic process where "name=\'node.exe\'" get CommandLine /value', { encoding: 'utf-8' });
if (result.includes('--inspect') || result.includes('--debug')) { process.exit(1); // 检测到调试模
式, 强制退出 } } catch (e) {} }, 60000); } `### 5.4 文件完整性校验 `typescript // 启动时校验关键
文件哈希, 防止篡改 import crypto from 'crypto'; const FILE_HASHES = { 'dist/app.js': '预期SHA256
值', 'license.lic': '预期SHA256值' }; for (const [file, expectedHash] of
Object.entries(FILE_HASHES)) { const content = fs.readFileSync(file); const hash =
crypto.createHash('sha256').update(content).digest('hex'); if (hash !== expectedHash) {
console.error(文件 ${file} 已被篡改, 系统拒绝启动); process.exit(1); } } ``
```

六、法律保护

措施	说明
软件著作权登记	在国家版权局登记, 获得法律保护依据
商业秘密协议	与客户签订NDA, 明确源码保密义务
软件许可协议	在系统中嵌入EULA, 明确使用范围和禁止事项
技术措施声明	在系统启动界面注明"本软件已采取技术保护措施"

七、实施路线图

第一阶段：基础防护

- ☒ 关闭 Source Map
- ☐ 前端代码混淆 (javascript-obfuscator)
- ☐ 更换数据库 sa 账号为最小权限账号
- ☐ 数据库连接启用加密

第二阶段：许可证机制

- ☐ 实现机器指纹采集
- ☐ 实现 License 签发工具 (RSA 非对称签名)
- ☐ 实现在线验证 + 离线验证双模式
- ☐ Express 全局中间件集成
- ☐ 前端 License 状态展示与到期提醒

第三阶段：深度防护

- ☐ 核心模块 bytenode 字节码编译
- ☐ Node.js 应用 pkg 打包
- ☐ 文件完整性校验

☐ 防调试检测

第四阶段：架构升级（可选）

- ☐ 核心算法服务化（SaaS）
- ☐ SQL Server TDE 加密
- ☐ 软件著作权登记

八、方案对比总结

方案	防护强度	实施成本	适用场景
前端混淆	★☆☆☆☆	低	所有项目必做
License 许可证	★★★★☆	中	所有部署项目必做
字节码编译	★★★★☆	中	核心模块保护
核心逻辑服务化	★★★★★	高	长期战略
数据库 TDE	★★★★☆	低	数据防泄露
法律保护	★★☆☆☆	低	必要补充

推荐优先级：License 许可证 > 前端混淆 > 字节码编译 > 数据库加固 > 核心服务化

文档版本：v1.0 | 编制日期：2026-05-06